

Unravelling **Abstract Cyclic Proofs** into **Proofs by Induction**

Lide Grotenhuis and Daniël Otten
ILLC, University of Amsterdam

Cyclic Proofs

Idea: replace (co)induction rules with sound circular reasoning.

Example. We have a cyclic proof of $x + 0 = x$ in arithmetic:

$$\begin{array}{c}
 \frac{}{\text{succ } x + 0 = \text{succ } (x + 0)} \text{ axiom} \quad x + 0 = x \\
 \vdots \\
 \frac{}{0 + 0 = 0} \text{ axiom} \quad \text{succ } x + 0 = \text{succ } x \\
 \hline
 x + 0 = x \quad \text{case}_x
 \end{array}$$

cycle

Might look unfamiliar, but proof assistants allow this:

$\text{unital} : (x : \mathbb{N}) \rightarrow (x + 0 = x),$
 $\text{unital } 0 := \text{refl},$ recursive call
 $\text{unital } (\text{succ } x) := \text{ap}_{\text{succ}}(\text{unital } x).$

Termination

Halting problem, so we pick a decidable termination condition:

- Structural Recursion (Lean, Rocq) (by default): every recursive call is to the same function and decreases the same inductive input.
- Size-Change Termination (Agda, Lambdapi, Idris, Isabelle/HOL): for every infinite sequence of recursive calls, we can eventually trace an inductive input that decreases infinitely often.

This condition is a lot more permissive:

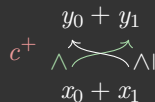
- mutually recursive functions,
- multiple inputs (lexicographic ordering, swapping, delayed tracking),
- easier to extend to mixed (co)induction.

Is it still sound?

Examples: Recursive Functions

Consider **addition** with a swapped recursive call:

$$\begin{aligned} 0 + x_1 &:= x_1, \\ (\text{suc } x'_0) + x_1 &:= \text{suc } \overbrace{(x_1 + x'_0)}^{c^+}, \end{aligned}$$



Or the **Ackermann** function:

$$\begin{aligned} A\ 0\ x_1 &:= \text{suc } x_1, \\ A\ (\text{suc } x'_0)\ 0 &:= \underbrace{A\ x'_0\ 1}_{c_0^A}, \\ A\ (\text{suc } x'_0)\ (\text{suc } x'_1) &:= \underbrace{A\ x'_0\ (\overbrace{A\ (\text{suc } x'_0)\ x'_1}^{c_1^A})}_{c_2^A}. \end{aligned}$$

c_0^A, c_2^A
 $\wedge \uparrow$
 $A\ x_0\ x_1$

c_1^A
 $\wedge \uparrow \uparrow \wedge$
 $A\ x_0\ x_1$

Both **not structurally recursive** but **size-change terminating**.

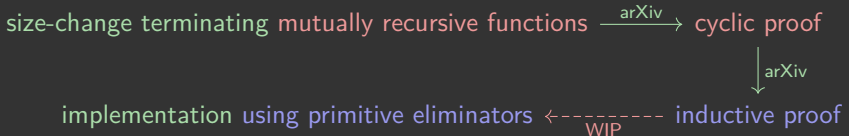
Compared with Induction

Both **Structural Recursion** and **Size-Change Termination** only accept terminating functions.

However, can we implement these functions using **primitive eliminators**?

- **Structural Recursion**: **Yes** (known):
 - implement **pattern matching** (with⁰ and without¹ axiom K),
 - one **induction** on the structural well-order for the **inductive input**.
- **Size-Change Termination**: **Work in progress**:
 - implement **pattern matching** using known techniques^{0,1},
 - do various **nested inductions** on **different functions** and **different inputs** in the **correct order**.

We approach this by proving an abstract **cyclic proof theory** result:



⁰Goguen, McBride, McKinna 2006

¹Cockx, Devriese, Piessens 2014

Overview

Unravelling **cycles** into **induction**:

- So far, this has been done for various **specific** proof systems⁰.
- We show how to do this in a **general** abstract way.
- In addition, we want to preserve the **structure** of the proof.

In particular:

- We start with an **abstract cyclic proof system** $\mathcal{C}$.
- We define an **induced (non-cyclic) proof system** $\mathcal{C}_{\text{ind}}$ that extends the rules of $\mathcal{C}$ with **intuitionistic well-founded induction**.
- We show how any **cyclic proof** π in $\mathcal{C}$ can be transformed into a **finite proof** π_{ind} in $\mathcal{C}_{\text{ind}}$ with the same **conclusion** and **structure**.

Our approach is greatly inspired by the PhD thesis of Dominik Wehr:

- Our main improvement is covering **non-linear inductive sorts**.

⁰arithmetic, modal/linear/first-order μ -calculus, inductive definitions, ...

Cyclic Proof System

Definition (cyclic proof system)

A *proof system* consists of:

- a set *Judg* (the judgments), a set *Rule* (the rule instances),
- for each rule $r : \text{Rule}$ an associated finite list of *premises* $\text{prem } r : \text{Judg}$ and a *conclusion* $\text{concl } r : \text{Judg}$.

A *cyclic proof system* is a proof system with additional structure:

- for each judgment $A : \text{Judg}$ a finite set of (trace) objects $\text{Ob } A$,
- for each rule $r : \text{Rule}$ and each premise index i a *size-change graph*

$$(\text{graph } r)_i : \text{Ob } (\text{concl } r) \rightarrow \text{Ob } (\text{prem } r)_i.$$

This is a bipartite graph where edges are labelled as either:

$$\geq \text{ (preserving) } \quad \text{or} \quad > \text{ (progressing).}$$

Proof

Definition (Proof)

A *derivation* is a (possibly infinite) tree where every node is labelled by a rule in a compatible way.

A *proof* is a *regular* derivation (it only has finitely many subderivations) such that for every *infinite branch*, we can eventually find a path of *trace objects* that has infinitely many *progressing* edges.

Every set of *mutually recursive functions* induces a *cyclic proof system*:

- for every function f we have a judgment $\downarrow f$ (read: f *terminates*) where the trace objects are the *inputs*, and a rule

$$\frac{\downarrow g \quad \text{for every call within } f \text{ to } g}{\downarrow f} f,$$

where $(\text{graph } r)_i : \text{Input } f \rightarrow \text{Input } g$ is given by the *structural order*.

- f is *Size-Change Terminating* iff the unique derivation of f is a *proof*.

Cyclic Representation

Proofs can be **infinite** but **regular**, so we can represent them with **cycles**:

Example. The following **infinite proof** has a **cyclic representation**:



Inductive Proof System

Given a **cyclic proof system** $\mathcal{C}$ we define a (non-cyclic) proof system $\mathcal{C}_{\text{ind}}$. The judgements of $\mathcal{C}_{\text{ind}}$ are sequents $\gamma_0, \dots, \gamma_{n-1} \vdash \delta$ of formulas:

$$\phi, \psi, \dots := A\bar{x} \mid x > y \mid x \geq y \mid \phi \rightarrow \psi \mid \forall x \psi.$$

Here A is a judgment of $\mathcal{C}$, treated as a relation symbol of arity $\text{Ob } A$.

$\mathcal{C}_{\text{ind}}$ has intuitionistic $\rightarrow$ and $\forall$ rules, quasi-order rules for $>$ and $\geq$, and:

$$x' \text{ fresh} \frac{\Gamma, \forall x' (x > x' \rightarrow \phi[x'/x]) \vdash \phi}{\Gamma \vdash \forall x \phi} >_{\text{ind}_x}.$$

Lastly, for every $\mathcal{C}$ -rule

$$\frac{A_i \quad \text{for every } i < n}{A} r,$$

with size-change graphs $(g_i : A \rightarrow A_i)_{i < n}$, we have a $\mathcal{C}_{\text{ind}}$ -rule

$$\bar{y} \text{ fresh} \frac{\Gamma, \bar{x} \succsim_{g_i} \bar{y} \vdash A_i \bar{y} \quad \text{for every } i < n}{\Gamma \vdash A \bar{x}} r.$$

Here $\bar{x} \succsim_{g_i} \bar{y}$ contains (strict/non-strict) inequalities according to g_i .

General Idea

We will transform a (possibly infinite) $\mathcal{C}$ -proof π of A_0 into a finite $\mathcal{C}_{\text{ind}}$ -proof π_{ind} of $\vdash A_0 \bar{x}_0$:

- We first translate π into a (possibly infinite) derivation $\pi_{\text{ind}}^{\text{inf}}$ in $\mathcal{C}_{\text{ind}}$:

$$\begin{array}{c}
 \vdots \\
 \hline A_3 \\
 r_2 \\
 \hline A_2 \\
 r_1 \\
 \hline A_1 \\
 r_0 \\
 \hline A_0
 \end{array}
 \rightsquigarrow
 \begin{array}{c}
 \vdots \\
 \hline \overline{x_0} \succsim_{g_0} \overline{x_1}, \overline{x_1} \succsim_{g_1} \overline{x_2}, \overline{x_2} \succsim_{g_2} \overline{x_3} \vdash A_3 \overline{x_3} \\
 \hline \overline{x_0} \succsim_{g_0} \overline{x_1}, \overline{x_1} \succsim_{g_1} \overline{x_2} \vdash A_2 \overline{x_2} \\
 \hline \overline{x_0} \succsim_{g_0} \overline{x_1} \vdash A_1 \overline{x_1} \\
 \hline \vdash A_0 \overline{x_0}
 \end{array}
 \begin{array}{l}
 r_2 \\
 r_1 \\
 r_0
 \end{array}$$

- We define π_{ind} by ‘cutting’ the infinite branches of $\pi_{\text{ind}}^{\text{inf}}$ short: we follow a nice **cyclic representation** π_{cyc} of π ; for each **cycle** we introduce an **induction hypothesis** below and apply it above.

A Simple Example

Consider the **swapped addition** function:

$$0 + x_1 := x_1,$$

$$(\text{suc } x'_0) + x_1 := \text{suc} \underbrace{(x_1 + x'_0)}_{c^+},$$

$$c^+ \quad \begin{array}{ccc} & y_0 + y_1 & \\ \nearrow & & \searrow \\ \wedge & & \vee \\ & x_0 + x_1 & \end{array}$$

The unique proof has (among others) the following two **cyclic representations**, where we have visualized the size-change graphs:



The second representation can be used to prune the infinite derivation $\pi_{\text{ind}}^{\text{inf}}$ into a finite proof π_{ind} (since there is an input with **progress**).

A Simple Example

The infinite derivation $\pi_{\text{ind}}^{\text{inf}}$ is:

$$\frac{\vdots}{\frac{x_0 > y_1, x_1 \geq y_0, y_0 > z_1, y_1 \geq z_0 \vdash \downarrow(z_0 + z_1)}{x_0 > y_1, x_1 \geq y_0 \vdash \downarrow(y_0 + y_1)} + \vdash \downarrow(x_0 + x_1)}$$

The cyclic representation of the previous slide yields π_{ind} :

$$\frac{\begin{array}{c} \text{(use } \text{hyp}[x_0] \text{ on } x'_0 := z_0 \text{ and } x'_1 := z_1) \\ \text{hyp}[x_0], x_0 > y_1, x_1 \geq y_0, y_0 > z_1, y_1 \geq z_0 \vdash \downarrow(z_0 + z_1) \\ \text{hyp}[x_0], x_0 > y_1, x_1 \geq y_0 \vdash \downarrow(y_0 + y_1) \end{array}}{\frac{\text{hyp}[x_0] \vdash \downarrow(x_0 + x_1)}{\vdash \downarrow(x_0 + x_1)} >_{\text{ind}'_{x_0}}}$$

where $\text{hyp}[x_0] := \forall x'_0 x'_1 (x_0 > x'_0 \rightarrow \downarrow(x'_0 + x'_1))$.

A Simple Example

The **cyclic representation** gives an **inlined swapped addition**:

$$\begin{array}{ccc}
 0 + x_1 := x_1, & & 0 + x_1 := x_1, \\
 (\text{succ } x'_0) + x_1 := \text{succ } (x_1 + x'_0), & \rightsquigarrow & (\text{succ } x'_0) + 0 := \text{succ } x'_0, \\
 & & (\text{succ } x'_0) + (\text{succ } x'_1) := \text{succ } (\text{succ } (x'_0 + x'_1)).
 \end{array}$$

We can implement the inlined definition using **primitive eliminators**:

$$\begin{aligned}
 x_0 + x_1 &:= \text{<-ind } x_0 \{ x_0 : \mathbb{N}, x_1 : \mathbb{N}, h : \Pi_{x'_0, x'_1} ((x_0 > x'_0) \rightarrow \mathbb{N}) \mapsto \text{inner}, \\
 \text{inner} &:= \text{case } x_0 \left\{ \begin{array}{l} 0 \mapsto x_1, \\ \text{succ } x'_0 \mapsto \text{case } x_1 \left\{ \begin{array}{l} 0 \mapsto \text{succ } x'_0, \\ \text{succ } x'_1 \mapsto \text{succ } (\text{succ } (h \ x'_0 \ x'_1 \ \text{ineq}_{x'_0}^{x'_0})). \end{array} \right. \end{array} \right.
 \end{aligned}$$

where **<-ind** and **case** are defined using **ind**.

Warning: The **inlined definition** and **primitive implementation** do not reduce $(\text{succ } x'_0) + x_1$ to $\text{succ } (x_1 + x'_0)$ but they **compute on closed terms**.

Homework: Implement **swapped addition** while preserving full computational behaviour or prove that this is impossible.

Conclusion

Theorem

*For every $\mathcal{C}$ -proof π of A , there exists a $\mathcal{C}_{\text{ind}}$ -proof π_{ind} of $\vdash A(\bar{x}_0)$.
Moreover, π_{ind} preserves the structure of π .*

Future work:

- Work out the application to **size-change termination**:
 - check preservation **computational behaviour**,
 - extend to **indexed inductive types**.
- Extend to a mix between **inductive** and **coinductive** sorts:
 - either by allowing both **inductive** and **coinductive** traces,
 - or through **ordinal approximations** or **sized types**.
- See whether the results hold in a **constructive metatheory**.

Preprint



<https://arxiv.org/abs/2602.12054>

Size-change Graph

Definition (size-change graph)

A *size-change graph* $G : X \rightarrow Y$ is a bipartite graph from a finite set X to a finite set Y where edges are labelled as either:

$\geq$ (preserving) or $>$ (progressing).

A *trace* through a sequence of size-change graphs $(G_i : X_i \rightarrow X_{i+1})_i$ consists of a starting time $k \in \mathbb{N}$ and nodes $(x_i \in X_i)_{i \geq k}$ such that x_i and x_{i+1} are connected by an edge in G_i .

We call an infinite sequence *progressing* if there exists a trace where the connecting edge is *progressing* infinitely often.

Examples:



(Cyclic) Heyting Arithmetic

Example. Robinson Arithmetic (Q) is intuitionistic first-order logic with function symbols $0, \text{suc}, +, \times$, the following axioms

$$\begin{array}{ll} 0 + y = y, & \text{suc}(x') + y = \text{suc}(x' + y), \\ 0 \times y = 0, & \text{suc}(x') \times y = (x' \times y) + y, \\ 0 \neq \text{suc}(y') & \text{suc}(x') = \text{suc}(y') \rightarrow x' = y', \end{array}$$

and an additional rule:

$$\frac{(\Gamma \vdash \delta)[0/x] \quad (\Gamma \vdash \delta)[\text{suc}(x)/x]}{\Gamma \vdash \delta} \text{case}_x$$

Heyting Arithmetic (HA) is Q extended with induction.

Cyclic Heyting Arithmetic (CHA) is Q as a cyclic proof system: the trace objects are the free variables, variable x progresses iff it passes case_x .

(Cyclic) Heyting Arithmetic

Our results can be used to transform a **CHA** (cyclic Heyting arithmetic) proof into a normal **HA** (Heyting arithmetic) proof:

- First we translate the **CHA-proof** into a **CHA_{ind}-proof**.
- Because **HA** and **CHA** are already first-order theories containing intuitionistic rules for $\rightarrow$ and $\forall$, and we can define

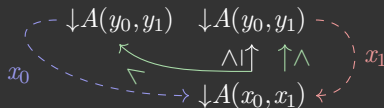
$$(n \leq m) := \exists x (n + x = m), \quad (n < m) := (\text{succ}(n) \leq m),$$

we can translate the **CHA_{ind}-proof** into a **HA-proof**.

The same strategy gives a transformation from **CPA** into **PA**.

Importance of Induction Order

The **termination proof** of the **Ackermann** function has **cyclic repr**:



We introduce an **induction hypothesis** for x_0 and x_1 in order:

$$\begin{array}{c}
 \frac{(\text{use } \text{hyp}_0(x_0) \text{ on } x'_i := y_i)}{\dots, x_0 > y_0 \vdash \downarrow A(y_0, y_1)} \quad \frac{(\text{use } \text{hyp}_1(x_1) \text{ on } x'_i := y_i)}{\dots, x_0 \geq y_0, x_1 > y_1 \vdash \downarrow A(y_0, y_1)} \\
 \hline
 \frac{\text{hyp}_0(x_0), \text{hyp}_1(x_1) \vdash \downarrow A(x_0, x_1)}{\text{hyp}_0(x_0) \vdash \downarrow A(x_0, x_1)} >_{\text{ind}'_{x_1}} \\
 \hline
 \frac{\text{hyp}_0(x_0) \vdash \downarrow A(x_0, x_1)}{\vdash \downarrow A(x_0, x_1)} >_{\text{ind}'_{x_0}}, \quad A
 \end{array}$$

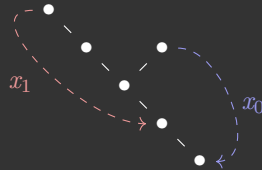
where the **induction hypotheses** are:

$$\text{hyp}_0(x_0) := \forall x'_0 \forall x'_1 (x_0 > x'_0 \rightarrow \downarrow A(x'_0, x'_1)),$$

$$\text{hyp}_1(x_1) := \forall x'_0 \forall x'_1 (x_1 > x'_1 \rightarrow \text{hyp}_0(x'_0) \rightarrow \downarrow A(x'_0, x'_1)).$$

Importance of Forgetting

Consider the following two forms of **cyclic representations**:



For both, if we want to transform them into finite $\mathcal{C}_{\text{ind}}$ -proofs, we have to introduce an induction hypothesis at the root for x_0 :

- in the first example, we can **forget** the induction hypothesis for x_0 (using **weakening**) before introducing the induction hypothesis for x_1 ;
- in the second example this is **not possible**, so in this example the back-edge for x_1 has to **'preserve'** x_0 in some way.

Non-Linear

The main technical difficulty is finding a suitable **cyclic representation**:

- We use annotations based on 'reset proofs' and the 'Safratov construction' to find a cyclic representation with an **induction order**.
- We use a technique of Sprenger and Dam to unfold the tree until the **order of sprouts** in the tree **respects** the **induction order**.

Here we **modify** reset proofs in one important way:

- Instead doing a reset whenever we have a '**cover**', we wait until we have a '**uniform cover**'.

This allows us to **remove** the restriction that the well-founded relation $\leq$ is a **linear order**: we only assume that it is a **quasi-order**.